

Forrajeo multi-agente con memoria externa en grillas: una historia experimental

Jeremías Figueiredo Paschmann

Francisco Nattero

Juan Kaplan

Proyecto final de Aprendizaje por Refuerzo

Resumen

Este informe resume una serie de experimentos de aprendizaje por refuerzo multi-agente en un entorno de forrajeo. El objetivo era entrenar hormigas locales que encontraran comida, la llevaran al hub y, eventualmente, usaran una memoria externa discreta para coordinarse. La historia experimental no fue lineal. El principal progreso no vino de “más comunicación” por sí sola, sino de definir correctamente la tarea, hacer que la señal de crédito fuera aprendible, aumentar la cobertura con más agentes y cambiar la arquitectura del crítico centralizado. En 25x25 se obtuvo una política que resuelve el umbral de 23 entregas bajo movimiento muestreado. En 50x50 la mejora más importante coincide con el paso de un crítico MLP plano a un crítico espacial `strided_cnn`; dentro de esa familia se llegó a resultados fuertes pero todavía confundidos con más hormigas, selección de puntos guardados, escritura compartida y penalizaciones de escritura. También hubo ramas que no deben desaparecer de la historia: autocurrículos que avanzaban sin entrega real, un flujo de laberintos sin resultado preservado comparable y barridos de memoria que aumentaban escritura sin probar comunicación útil. En 250x250 apareció una lección negativa: el retorno moldeado puede crecer aunque las entregas reales sean cero. La conclusión honesta es que el sistema aprendió forrajeo multi-agente útil y escaló mejor de lo esperado, pero todavía no demuestra causalmente comunicación emergente mediante bytes.

1. Motivación

El problema que se estudia es simple de describir y difícil de aprender: varios agentes locales deben encontrar fuentes de comida, recoger unidades de comida y volver al hub para entregarlas. Cada agente observa solo una vecindad pequeña y ejecuta una política compartida. Durante el entrenamiento se permite un crítico centralizado, pero durante el despliegue la política sigue siendo local. Esta separación corresponde al marco de entrenamiento centralizado con ejecución descentralizada, usado en algoritmos multi-agente como MAPPO y MADDPG [6, 2].

La motivación no es solo maximizar una métrica. La pregunta interesante es semántica: qué comportamiento aparece cuando se combinan búsqueda local, memoria en el ambiente y presión de entregar comida. Un resultado útil no debería ser únicamente “alto reward”. Debería mostrar ciclos completos de forrajeo, regreso al hub, uso razonable del espacio y robustez bajo mapas aleatorios. Por eso este informe distingue tres cosas que a menudo se mezclan:

- desempeño: cuánta comida se entrega, con qué tasa de éxito y con cuántos pasos;
- mecanismo: qué cambios de entorno, política o crítico hicieron posible el desempeño;
- comportamiento: si los videos muestran búsqueda, transporte y retorno, o solo movimiento con retorno moldeado.

El trabajo se ubica cerca de MAPPO [6], PPO [4], GAE [3] y aprendizaje de comunicación multi-agente como CommNet y DIAL [5, 1]. La diferencia práctica es que aquí la comunicación

no es un canal diferenciable directo entre agentes. Es una memoria discreta escrita en celdas del ambiente. Eso hace que la atribución causal sea más difícil: ver bytes no-cero no implica que esos bytes estén coordinando la política.

2. Entorno y semántica

La semántica del entorno cambió de manera decisiva durante el proyecto. En la versión relevante para este informe, `food_count` no significa posiciones independientes de comida. Significa mordidas totales. `food_sources` controla cuántas fuentes concentran esas mordidas y cada fuente se agota. La recompensa cruda principal es la entrega en el hub: una unidad entregada da +1. La recogida puede tener bonus de moldeado en algunas corridas, pero no es el objetivo final.

Esta distinción cambia la interpretación de todos los resultados. Un resultado de 23/23 en 25x25 no significa visitar 23 puntos aislados. Significa completar 23 entregas desde 6 fuentes agotables. Cambiar `food_sources` con `food_count` fijo cambia la geometría de la tarea, la repetición de rutas y la dificultad de redescubrir fuentes.

Tabla 1: Variables semánticas que no deben tratarse como detalles menores.

Variable	Interpretación para el informe
<code>food_count</code>	Total de unidades que pueden ser entregadas. Es el denominador natural de la fracción entregada.
<code>food_sources</code>	Número de fuentes agotables. A igual <code>food_count</code> , menos fuentes implican fuentes más densas.
<code>sampledmovement</code>	Despliegue estocástico de movimiento. Muchas políticas buenas viven en la distribución, no en el argmax.
<code>greedymovement</code>	Despliegue determinista. Es más estricto y frecuentemente empeora el retorno.
<code>critic_architecture</code>	Arquitectura del crítico centralizado. Cambia la señal de valor durante PPO aunque el actor local no reciba observación global en ejecución.
<code>write_while_moving</code>	Permite escribir mientras se mueve. Cambia el costo y la frecuencia efectiva de escritura.

3. Método

La base de entrenamiento es PPO multi-agente con política compartida, ventajas estimadas con GAE y un crítico centralizado. PPO optimiza una función objetivo recortada que evita cambios demasiado grandes de política [4]. MAPPO aplica esta idea a agentes cooperativos y suele usar un crítico centralizado para estabilizar el aprendizaje [6]. En este proyecto, el actor debe ejecutar con información local; el crítico puede ver más información durante entrenamiento.

La arquitectura del crítico es una frontera causal del proyecto:

- `mlp`: crítico plano sobre observaciones centralizadas aplanadas. Fue usado en líneas tempranas y en el 50x50 eficiente inicial.
- `strided_cnn`: crítico espacial sobre planos de grilla y rasgos de entidades. Aparece en la línea 50x50 de proximidad y full-layout.
- `set_cnn`: familia posterior usada en diagnósticos 250x250. No es una continuación limpia de los experimentos 50x50.

Esta separación es central. Si una corrida 50x50 mejora después de pasar de `mlp` a `strided_cnn`, no se puede atribuir la mejora solamente a más hormigas, más bits, fuentes más densas o escritura compartida. El crítico cambió el problema de crédito que PPO resuelve durante entrenamiento.

Cada corrida se leyó como un conjunto de artefactos, no como una captura aislada. El contrato de tarea queda en `experiments/*.json` o en el notebook que arma la corrida. La traza temporal queda en `metrics.jsonl`. El cierre queda en `summary.json`. Los pesos quedan en checkpoints finales y, cuando se activó selección, en checkpoints “best”. Los videos de W&B o locales sirven como evidencia cualitativa sincronizada con updates o checkpoints, pero no reemplazan las métricas de tarea.

Esta trazabilidad importa porque “best checkpoint” no significa “último checkpoint”. El runner puede seleccionar por una métrica de entrenamiento o de evaluación, en modo máximo o mínimo, y las evaluaciones pueden usar semillas desplazadas, layouts barajados y modos de acción como movimiento muestreado con escritura greedy. Por eso el informe separa cuatro niveles: configuración, curva de entrenamiento, evaluación seleccionada y video. El sandbox del sitio web debe leerse igual: exporta el actor 50x50 a JavaScript, sin crítico, sin JAX y sin entrenamiento. Es una herramienta de inspección de la política local, no una nueva corrida de entrenamiento.

Tabla 2: Cómo se interpreta una corrida en este informe.

Artefacto	Qué fija	Riesgo si se omite
<code>config.json</code> o <code>experiments/*.json</code>	Tamaño, fuentes, comida, hormigas, bits, radio, crítico, horizonte, rewards y modo de transferencia.	Comparar corridas que no tienen el mismo contrato.
<code>metrics.jsonl</code>	Evolución por update: entregas, pickups, cobertura, bytes, entropía, KL, gradientes y pérdidas.	Confundir un pico, una caída o una métrica moldeada con desempeño final.
<code>summary.json</code>	Estado final y, cuando existe, métricas del mejor checkpoint.	Mezclar último checkpoint con checkpoint seleccionado.
Videos locales / W&B	Comportamiento visible: búsqueda, recogida, regreso al hub, saturación de bytes o movimiento sin entrega.	Usar un video vistoso como sustituto de eventos reales.

Las figuras siguen un criterio editorial simple: el lienzo del gráfico se reserva para datos, ejes, leyendas, umbrales y anotaciones numéricas. La interpretación de cada resultado aparece en la leyenda y en el texto que acompaña a la figura, no como un título incrustado dentro del gráfico.

4. Historia experimental

4.1. Cambio de contrato de la tarea

El primer cambio grande fue conceptual. La tarea dejó de premiar fuertemente recoger comida y pasó a medir entregas al hub con fuentes agotables. Esto convirtió el problema en forrajeo real: encontrar no basta, hay que cerrar el ciclo fuente-hub. También hizo que la densidad de fuentes fuera una variable de dificultad.

Esta frontera invalida comparaciones ingenuas con corridas viejas. Un reward alto bajo pickup reward no significa lo mismo que un reward alto bajo delivery-only reward.

4.2. Primer enfoque: mapas cada vez más grandes con un solo agente

La primera línea JAX intentó resolver el problema como un currículo de tamaño: entrenar en mapas chicos y aumentar progresivamente hasta 50x50. El contrato de `forage_curriculum.json` era una hormiga, radio local 1, 48 unidades de comida repartidas en 12 fuentes, 1 bit de escritura, 10000 pasos máximos, `hidden_size=128` y etapas explícitas de 4x4 a 50x50. Era una idea razonable: si la política aprende el ciclo fuente-hub en chico, tal vez pueda transferirlo a mapas más grandes.

No alcanzó. El reporte consolidado local registra que el retorno bajaba de 12,281 en 4x4 a 0,719 en 25x25 y 0,156 en 50x50. El patrón observado era compatible con agentes que exploran o cargan comida, pero no convierten eso en ciclos repetidos de entrega.

La causa más plausible no fue falta de bits. Era una mezcla de horizonte largo, recompensa escasa, cobertura insuficiente y crédito débil.

4.3. El desbloqueo 25x25

El primer resultado fuerte aparece al combinar moldeado de distancia, más hormigas y más capacidad. `DISTANCE_SHAPE` ya hacía parcialmente aprendible el umbral 25x25: 13,75/23 en greedy, pero solo 7/23 muestreado. `DISTANCE_CAP4` cambia a cuatro hormigas y hidden size mayor; allí la evaluación muestreada llega a 23/23 con éxito 1,0, aunque el modo greedy cae a 2,75/23.

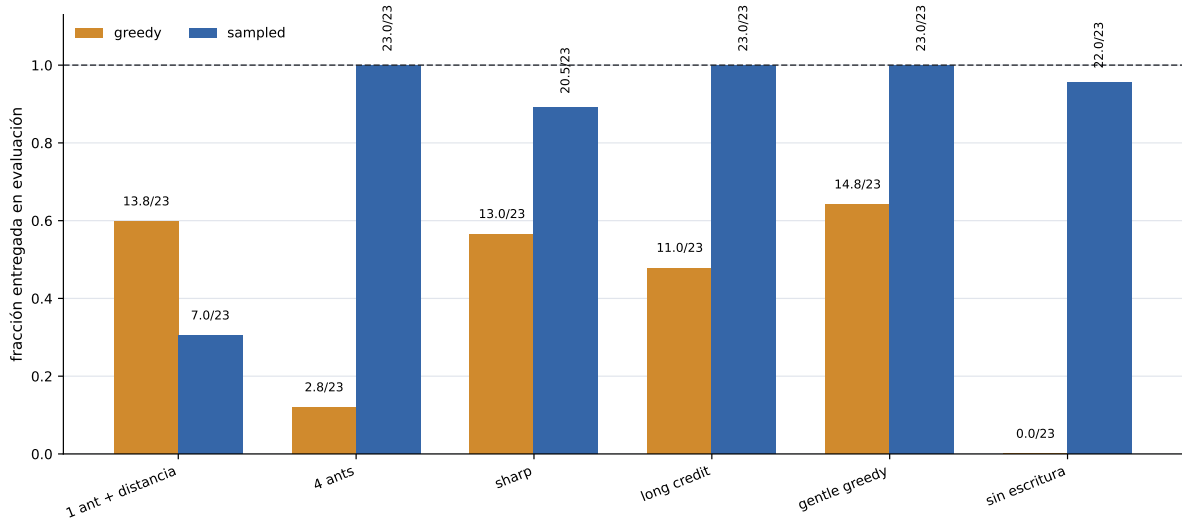


Figura 1: Evaluación 25x25 bajo dos protocolos de despliegue. Cada barra muestra la fracción de comida entregada y anota entregas sobre 23 unidades. La línea punteada marca la tarea completa: el resultado importante es la brecha entre movimiento greedy y movimiento muestreado, no solo el máximo alcanzado.

El resultado `DISTANCE_CAP4_NO_WRITE` es una advertencia contra sobreinterpretar comunicación: incluso sin escritura, una continuación entrega 22/23 bajo movimiento muestreado. No es una ablación perfectamente controlada, porque también cambian schedule y optimizador, pero sí muestra que la solución 25x25 no requiere demostrar comunicación por bytes.

Las continuaciones de 25x25 muestran que el problema no era un solo interruptor. `DISTANCE_CAP4_SHARP` mejora greedy a 13/23, pero baja el resultado muestreado a 20,5/23. La continuación long-credit recupera el 23/23 muestreado, aunque deja greedy en 11/23. La variante gentle-greedy sube greedy a 14,75/23 y conserva el 23/23 muestreado. `DISTANCE_VISION2_CAP4` también logra 23/23 muestreado con radio 2, pero cambia observación y capacidad. La lectura no es que

una ablación aislada resolvió todo; es que shaping, cobertura, capacidad y modo de despliegue formaron un paquete aprendible.

4.4. Bits contra cobertura

Los barridos tempranos de bits no muestran un salto claro de capacidad. En una línea 15x15 aproximada, el retorno de entrenamiento sube de 6,40 a 7,36 al pasar de 2 a 5 bits y luego baja levemente en 8 bits. En la línea anclada a 25x25, el retorno se mantiene bajo. En cambio, aumentar el número de hormigas en 25x25 con 3 bits produce una mejora monotónica de retorno de entrenamiento: 7,7875 con 2 hormigas, 12,545 con 4 y 18,2294 con 8.

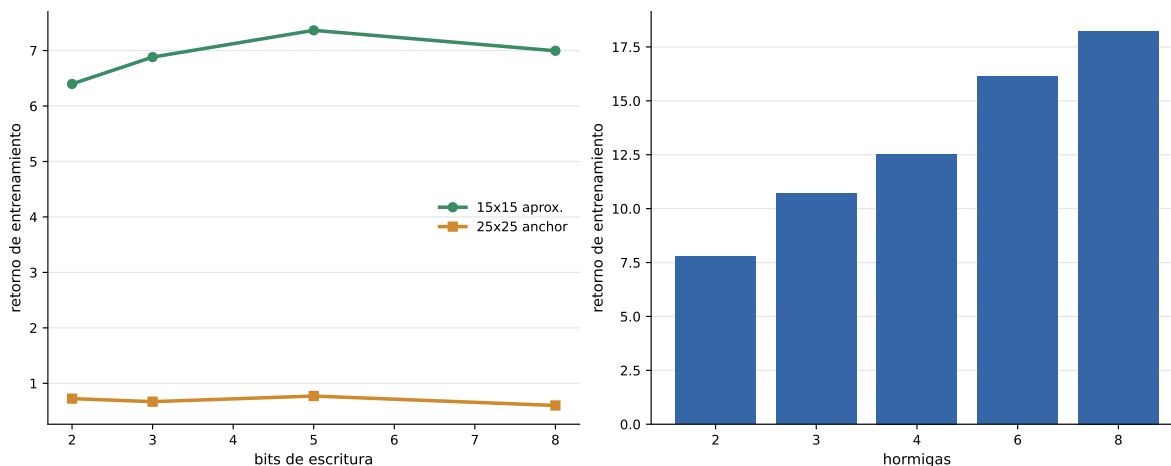


Figura 2: Comparación entre dos hipótesis tempranas. El panel izquierdo muestra retornos de entrenamiento al variar la cantidad de bits de escritura; el panel derecho muestra retornos al variar la cantidad de hormigas. La lectura favorece una explicación de cobertura multi-agente antes que una explicación basada solo en aumentar el alfabeto de escritura. Estas curvas no son evaluaciones held-out.

4.5. Métricas registradas más allá del retorno final

Las corridas locales y los resúmenes W&B no registraban solo reward final. En varias ramas se guardaron `delivery_events`, `pickup_events`, `mean_carrying_ants`, cobertura visitada, fracción de bytes no-cero, tasa de escritura, entropía, `approx_kl`, norma de gradiente, `value_loss` y, cuando había evaluación, comida entregada por 1000 pasos-hormiga. Esa instrumentación es parte de la historia: permitió detectar políticas que exploraban casi todo el mapa, escribían mucho o cargaban comida, pero no necesariamente entregaban.

En la línea 50x50 larga, por ejemplo, la cobertura final sube desde 0,059 al principio hasta 0,9275 hacia el final, mientras la tasa de escritura no-cero baja desde 0,113 hasta 0,0168. En la línea eficiente 50x50, los logs de evaluación registran eficiencia: una evaluación temprana dio 13,5 entregas y 0,54 entregas por 1000 pasos-hormiga; otra alcanzó 18,0 y 0,72, aunque más tarde esa familia volvió a oscilar. Estos números explican por qué el informe no debe contar solo el mejor video.

4.6. Autocurrículos que no cerraron el ciclo

El proyecto probó autocurrículos de dos tipos. El primero era un entorno 50x50 acolchado donde la grilla activa empezaba chica y crecía después de cumplir un umbral de entregas. La idea era razonable: entrenar una única política contra observaciones de tamaño fijo, pero exponerle dificultad progresiva. En la práctica, esta rama quedó por debajo del desbloqueo por cobertura y moldeado de distancia. El contrato relevante era una política de una hormiga, 12 unidades de

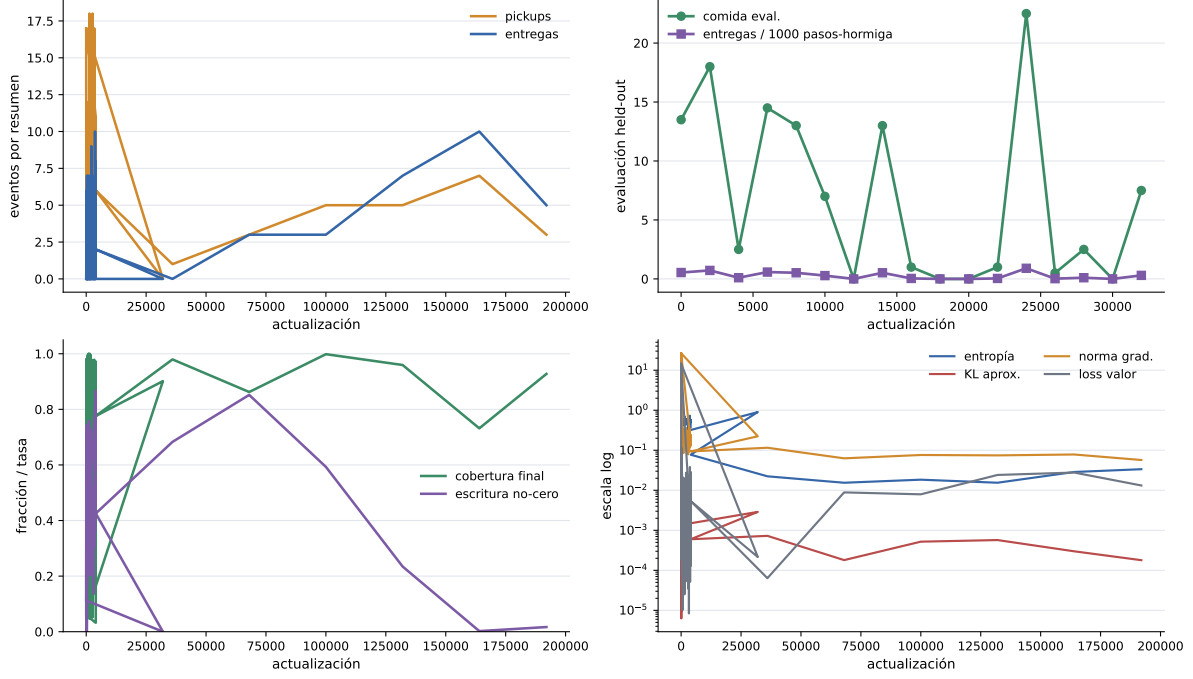


Figura 3: Métricas registradas en logs locales de dos ramas 50x50. Los paneles separan eventos de tarea, evaluación held-out, cobertura/escritura y señales de optimización. La figura no reemplaza las tablas finales: muestra por qué el análisis necesita más dimensiones que retorno final o apariencia visual.

comida, 2 fuentes, 1 bit de escritura, radio de visión 1, crítico MLP y grilla activa 4x4–50x50. El reporte archivado muestra que el autocurrículo aumentó el tablero sin producir competencia transferible: retorno completado 12,281 en 4x4, 0,719 en 25x25 y 0,156 en 50x50.

El segundo tipo fue más importante como diagnóstico: autocurrículo de distancia en 250x250 con 500 hormigas, 5000 comidas, una fuente rica, 4 bits, radio 2 y crítico `set_cnn`. Allí el problema no era que “no pasara nada”. Pasaba demasiado: retorno moldeado, agentes cargando, bytes activos y cambios de etapa podían coexistir con entrega real nula. Una corrida resumida reporta `delivery_events=0`, `pickup_events=0`, `env_return=0`, `episode_return=14.641`, cuatro etapas completadas hasta distancia 32, `mean_carrying_ants=447` y `final_mean_nonzero_byte_fraction=0.862`. Una diagnosis posterior muestra `delivery_events=0`, `pickup_events=0`, `shaping_return=26.25`, dos etapas completadas hasta distancia 8, `mean_carrying_ants=257` y fracción final de bytes no-cero 0.899.

La lectura es metodológica: los umbrales de un autocurrículo no pueden confiar solo en progreso moldeado, actividad de bytes o cantidad de agentes cargando. Deben depender de eventos de tarea, especialmente entregas y tasa pickup-entrega. Este fracaso motivó reportar `pickup_to_delivery_rate`, `undelivered_pickup_events`, ventanas sin entrega y evaluaciones `normal/no_byte_read/no_write`. También explica por qué la recuperación `reset-boundary` de 250x250 debe presentarse como cambio de contrato de reset y distancia, no como prueba de que el autocurrículo general quedó resuelto.

4.7. Laberintos: una rama de infraestructura sin resultado comparable

Otra rama exploró laberintos. En la rama actual queda el config `maze_exploration_curriculum.json`: laberintos generados de 10x10 a 50x50, una hormiga, 48 unidades de comida, 12 fuentes, 2 bits, radio 1, crítico MLP, `max_steps=6250`, 16 entornos por 80 pasos, `reward_mode=explore`, pasillos de ancho 3, paredes de ancho 1 y semilla 17. La infraestructura de obstáculos sí quedó implementada y testeada: obstáculos bloquean movimiento, aparecen en

observaciones, los resets ubican hub/comida/hormigas en celdas abiertas y los layouts pueden cambiar tras truncación.

También existió una rama lateral `origin/lethal_cookies`. Allí el cuaderno de maze preparaba una prueba de estrés 41x41 en forma de L dentro de una arena 50x50, con pasillos de ancho 5, 20 hormigas, una fuente normal al final y, en una variante hermana, bolsillos laterales para fuentes letales. Sin embargo, esa rama no tiene en el checkout actual métricas locales preservadas de entrega, supervivencia, eficiencia o convergencia, y la llamada de entrenamiento del cuaderno de maze estaba comentada.

Por eso el laberinto debe contarse como fracaso de flujo/evidencia, no como fracaso cuantitativo de política. La idea era válida para probar memoria espacial bajo obstáculos, pero no produjo una corrida comparable con las ramas principales. La decisión experimental fue volver al forrajeo abierto, donde sí existían curvas, videos, puntos guardados y métricas de entrega.

4.8. Memoria y autoresearch: escribir no basta

La línea de memoria intentó convertir la observación cualitativa de bytes escritos en una afirmación causal. El criterio correcto era más exigente: una política con bytes debía superar pruebas `no_byte_read` y `no_write`, y al mismo tiempo mantener o mejorar comida entregada. Los barridos R8–R12 y las recompensas `delivery_byte_trail_bonus`, `byte_follow_bonus` y variantes relacionadas fueron diseñados para esa pregunta.

La conclusión no fue positiva. El punto guardado sparse apenas dependía de los bytes. R8 entregó bien pero no a través de memoria: `normal`=394, `no_byte_read`=397 y `no_write`=397. R9 produjo una brecha más causal, pero dañó el forrajeo: `normal`=124 contra ablaciones de 56. R10 fue más liviano, aunque todavía débil: 260 contra 248. R11 preservó conducta sparse sin ganancia causal: 378 contra 395 y 395. R12 quedó interrumpido antes de un resultado final confiable. Junto con el resultado `DISTANCE_CAP4_NO_WRITE` de 22/23, esto impide afirmar que la memoria externa causó los mejores resultados. Lo que sí queda demostrado es más modesto: el entorno ofrece una memoria discreta y las políticas pueden escribirla; falta probar que esa información escrita mejore decisiones de forma robusta.

4.9. Fuentes cada vez más dispersas

Otra línea importante no escalaba por tamaño de mapa, sino por geometría de comida. La hipótesis era que el cuello de botella podía ser descubrimiento de fuentes: si la comida empieza fácil de encontrar y luego se vuelve escasa, quizá la política aprenda búsqueda y explotación sin depender tanto de ayudas de distancia.

Hubo varias formas de probarlo. `exploration_to_forage_padded_sources_50x50` mantuvo una arena 50x50 por compatibilidad de checkpoint, restringió hub y comida a una ventana interior 20x20, partió de un checkpoint 20x20 y redujo posiciones de fuente de 12 a 2 con 18 unidades totales. Esa forma fue débil: alrededor de 0,25/18. `exploration_to_forage_scratch_smooth_sources_50x50` fue más ambiciosa: arena exterior 80x80, ventana interior 50x50, 250 unidades de comida, 4 bits y annealing suave de posiciones de fuente desde 250 hasta 2. No quedó como resultado principal. `exploration_to_forage_proximity_sources_50x50` cambió la idea: mantener dos macro-fuentes desde el inicio, pero empezar con footprints cercanos y reducirlos hasta dos celdas finales dentro de una ventana interior 30x30.

El resultado de proximidad llegó a 40,875/250, pero no debe leerse aislado: esa rama también introduce el crítico espacial `strided_cnn`. Por eso el currículo de fuentes dispersas es una pieza de la historia, no una conclusión causal cerrada. Mostró que descubrimiento de comida y representación espacial del crítico eran cuellos distintos.

Tabla 3: Currículos de tamaño y de fuentes dispersas.

Rama	Contrato	Lectura
<code>forage_curriculum</code>	1 hormiga, 4x4–50x50, 48 comidas, 12 fuentes, 1 bit.	Primer enfoque por mapas crecientes; colapsa al crecer.
<code>autocurriculum</code>	cuadrado activo 4x4–50x50, avance tras 6 entregas por etapa, 12 comidas en 2 fuentes.	Avanza etapas, pero no produce competencia transferible robusta.
<code>padded_sources</code>	50x50 oculto, ventana interior 20x20, posiciones de fuente 12–2, 18 comidas.	Resultado débil, cerca de 0,25/18.
<code>smooth_sources</code>	80x80 exterior, 50x50 interior, 250 comidas, posiciones 250–2.	Prueba de annealing suave; no queda como resultado principal.
<code>proximity_sources</code>	50x50 exterior, 30x30 interior, 4 hormigas, 250 comidas, dos macro-fuentes con footprint decreciente.	40,875/250, mezclado con <code>strided_cnn</code> .

4.10. Ablaciones y diagnósticos que cambiaron la interpretación

No todas las ramas fueron resultados principales, pero varias cambiaron qué se podía afirmar. La regla de lectura fue conservadora: cuando una corrida cambia muchas cosas a la vez, se cuenta como intervención; cuando compara una política contra una versión `no_write`, `no_byte_read` o un costo equivalente, se acerca más a una ablación causal.

Tabla 4: Diagnósticos que evitaron sobreinterpretar el resultado principal.

Diagnóstico	Cambio	Número clave	Lectura
<code>DISTANCE_CAP4_NO_WRITE</code>	25x25 sin escritura.	muestreado 22/23, éxito 0,5.	La política 25x25 no necesita demostrar comunicación por bytes para explicar el solve muestreado.
R8–R12 memoria	<code>normal</code> contra <code>no_byte_read</code> y <code>no_write</code> .	R8 394/397/397; R9 124/56/56.	La dependencia causal aparece cuando el forrajeo empeora; no prueba una memoria útil y robusta.
Escritura compartida y costo	8 hormigas 50x50 con escritura compartida y penalización.	45,25/125 a 69,375/125.	Progreso práctico en el linaje <code>strided_cnn</code> , no ablación aislada.
Sondeo de comunicación Costo de escritura x300 / 6 fuentes	Evaluación del checkpoint con costo de escritura. Costo severo y fuentes más numerosas.	determinista 18,5/125, muestreado 22,25/125. primer eval 68,375/125, final 19,625/125.	Los bytes existen, pero la lectura causal sigue débil. Reducir escritura puede degradar la política si el contrato cambia demasiado.
250x250 trunc4096	Continuación desde frontera con horizonte truncado.	mejor train 685, final 13.	Hubo picos, pero no estabilidad final.
Directa / exploración / laberinto	Líneas base e infraestructura lateral.	sin resultado local preservado comparable.	Se incluyen como decisiones de historia, no como evidencia de éxito.

4.11. 50x50 raro y ayudas vectoriales

Cuando la comida se vuelve rara en 50x50, el problema cambia. El agente debe descubrir fuentes con baja probabilidad y luego sostener ciclos. La línea base rara llega a 10,96/23. Agregar visión de radio 2 no ayuda. Agregar vector al hub mejora poco. Agregar vector al hub y a la comida cercana sube a 15,20/23, y selección held-out llega a 16,20/23 en la confirmación final. La rama posterior de radio de spawn fue inconclusa, no negativa: completó **spawn8** con retorno de entrenamiento 5.898, **spawn16** con 7.209 y se detuvo en 130/500 actualizaciones de la etapa final sin **summary.json**.

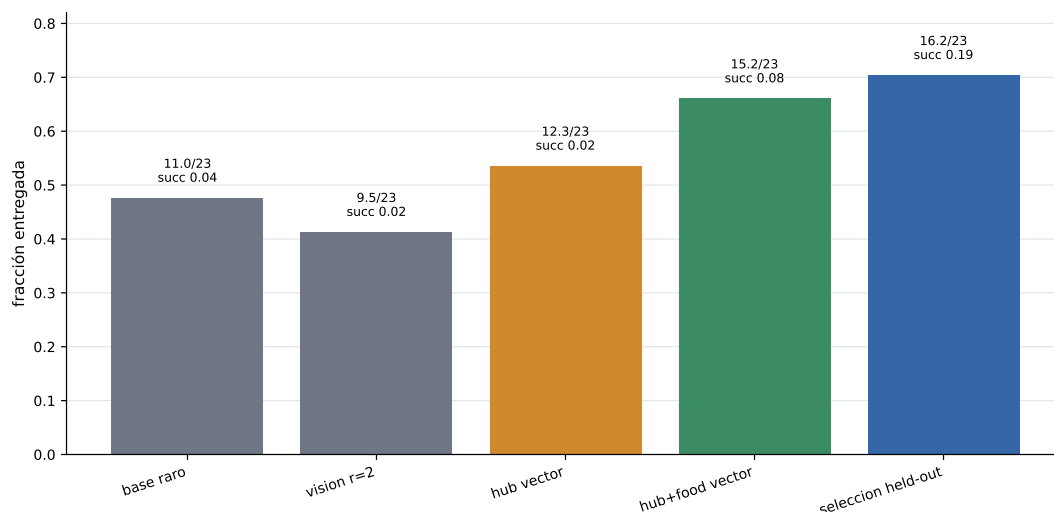


Figura 4: Evaluaciones seleccionadas en el escenario 50x50 con fuentes raras. Las barras reportan fracción entregada y anotan entregas sobre 23 junto con tasa de éxito. Las ayudas vectoriales parecen facilitar descubrimiento, pero la mejora no prueba comunicación emergente porque las corridas cambian más de una variable.

4.12. La frontera del crítico en 50x50

La línea 50x50 eficiente inicial usa el crítico MLP por defecto y entrega 21,5/48. La línea posterior de proximidad y full-layout cambia a **strided_cnn**. Ese cambio es mayor: el crítico ya no procesa una observación central plana, sino una representación espacial. Dentro de esa familia aparecen mejoras importantes, pero también colapsos de checkpoint y fuerte dependencia de selección.

El mejor resultado local de 8 hormigas en esa familia es **shared_writes_write_cost_from_shared_best**: 69,375/125, éxito 0,125. Es progreso real, pero no una solución completa. También tiene escritura no-cero muy alta, alrededor de 0,94, por lo que no alcanza para afirmar que haya un código compacto e interpretable.

La línea reciente de 60 hormigas es la frontera actual del repo. Una evaluación de 64 episodios del punto guardado estabilizado reporta 123,90625/125, fracción 0,99125, éxito 0,90625. Esto es fuerte como ingeniería de comportamiento 50x50, pero cambia demasiadas cosas a la vez: 60 hormigas, 8 bits, actor con identidades repetidas, continuación desde un punto guardado, crítico **strided_cnn**, selección por evaluación, temperaturas y escritura casi saturada. Debe presentarse como resultado prometedor, no como prueba causal de comunicación.

4.13. 250x250: la trampa del retorno moldeado

La escala 250x250 mostró el fracaso más informativo. Corridas con **resnet_cnn** o **strided_cnn** en una fuente terminan con cero pickups y cero entregas. En la familia **set_cnn**, el

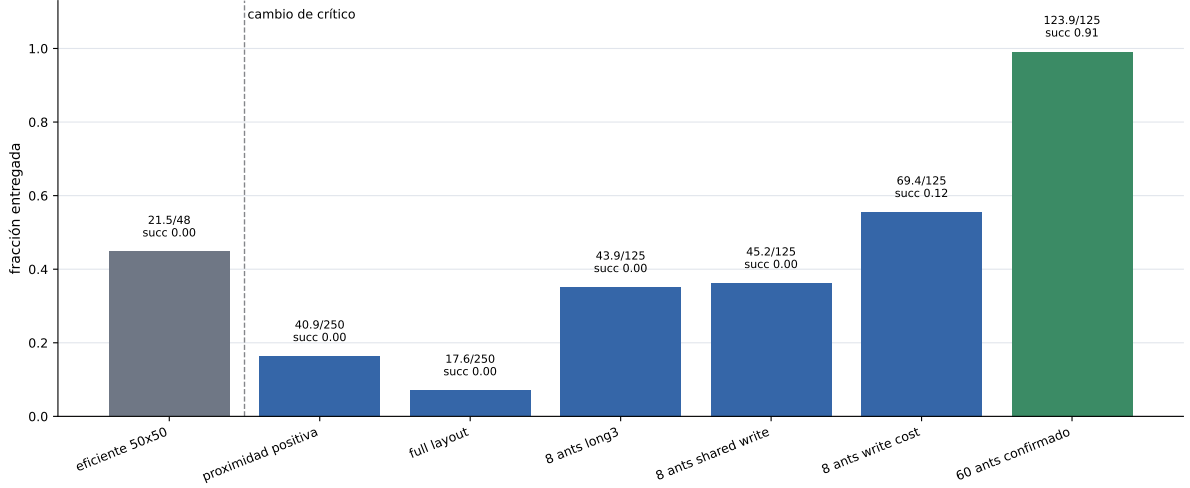


Figura 5: Resultados 50x50 seleccionados con denominadores explícitos en cada barra. La línea vertical marca el cambio de crítico MLP a crítico espacial. Las barras posteriores no son una sola ablación: combinan arquitectura del crítico, geometría de fuentes, cantidad de hormigas, escritura, costos y selección de checkpoints.

autocurrículo de distancia puede producir retorno positivo de moldeado, muchos agentes cargando y mapas de bytes saturados, pero todavía cero entregas finales. Es exactamente el tipo de resultado que un informe debe separar de la métrica objetivo.

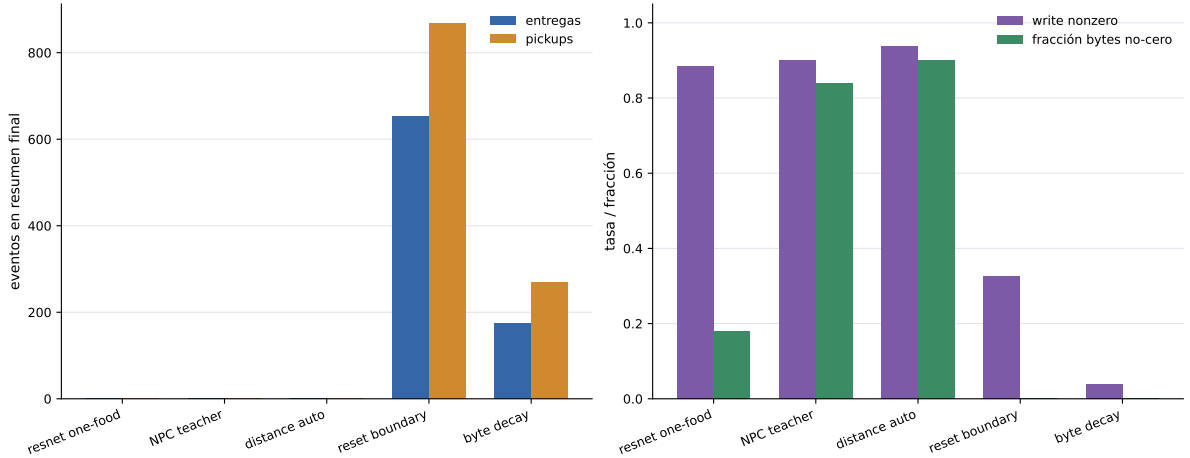


Figura 6: Diagnóstico 250x250 separado por métrica. El panel izquierdo muestra eventos reales de tarea, entregas y pickups. El panel derecho muestra actividad de escritura y fracción de bytes no-cero. La comparación deja visible la lección negativa: el retorno moldeado y la actividad de bytes pueden crecer sin resolver entrega; reset-boundary recupera progreso local de entrega.

El cambio `fixed8-reset-boundary256` sí mejora la entrega local: el resumen final tiene 654 entregas y 869 pickups, y el mejor tramo de entrenamiento llega a alrededor de 1003 entregas. La evaluación `fresh-window` del mismo linaje selecciona `reset_boundary_final` como campeón por mediana de entregas. Aun así, esto no resuelve la escala general. Es una intervención de reset y distancia dentro de una familia de crítico distinta.

Las continuaciones posteriores refuerzan esa lectura. `byte-decay` redujo ocupación de bytes pero terminó con 175 entregas. La evaluación larga `d12` de la rama `no-decay` mostró una media de 0.392 entregas, mediana 0, `pickup_to_delivery_rate`=0.008 y 13.745 pickups no entregados en promedio. El intento estratificado `d4/d8/d12` dejó manifiesto y configuración,

pero los archivos `metrics.jsonl`, `fresh_eval_metrics.jsonl` y `long_eval_metrics.jsonl` quedaron vacíos después de un aborto operacional tipo `exit 137`. No es evidencia contra la hipótesis; simplemente no produjo un resultado interpretable.

5. Evolución cualitativa del comportamiento

Los videos apoyan una lectura progresiva. Primero hay movimiento local sin retorno estable. Después, los rollouts preservados del currículo 25x25 muestran cómo cambia el comportamiento al pasar de 2 a 4 y 8 hormigas: la cobertura deja de ser un detalle y se vuelve parte del mecanismo. En 50x50 con crítico espacial aparecen rutas repetidas y entregas más densas. Las visualizaciones grandes deben leerse con cuidado: 100x100 es una continuación seleccionada, 250x250 muestra una recuperación local por `reset-boundary`, y 1000x1000 es un despliegue solo-actor de una política ya entrenada, no entrenamiento de punta a punta en ese entorno.

6. Qué funcionó y qué no

Funcionó, con evidencia fuerte:

- redefinir la tarea alrededor de entregas reales y fuentes agotables;
- usar moldeado de distancia para atravesar el horizonte largo inicial;
- aumentar cobertura con más hormigas;
- evaluar políticas muestreadas cuando el comportamiento aprendido vive en la distribución;
- cambiar el crítico centralizado a una arquitectura espacial para 50x50.

Funcionó parcialmente o con confusión causal:

- escritura compartida y penalización de escritura en 50x50;
- aumento a 8 bits, porque también cambió la escala de costo;
- línea de 60 hormigas, porque mejora mucho pero mezcla muchos factores;
- `reset-boundary` en 250x250, porque arregla ventanas locales pero no demuestra curriculum general.

No funcionó o no quedó demostrado:

- aumentar bits como único mecanismo de escala;
- los autocurrículos como solución general: podían avanzar o acumular retorno moldeado sin entrega real;
- el flujo de laberintos como evidencia empírica comparable: hubo implementación y tests, pero no una corrida preservada con métricas finales;
- las ramas R8–R12 de memoria como prueba de comunicación: algunas aumentaron dependencia de bytes, pero no mejoraron simultáneamente la tarea;
- despliegue determinista como criterio único de éxito;
- interpretar retorno moldeado como entrega real;
- afirmar comunicación emergente causal sin ablaciones emparejadas de `no-write`, `shuffled-write` y costo de escritura.

7. Limitaciones

La limitación principal es experimental: muchas corridas son continuaciones con varios cambios simultáneos. Por eso este informe habla de “intervenciones” y no de ablaciones puras salvo donde haya evidencia comparable. También hay diferencias entre punto guardado final, mejor punto guardado, selección held-out y resúmenes W&B. Esas diferencias importan porque varias corridas tienen pico temprano y luego degradación.

La segunda limitación es semántica. La memoria externa existe y se usa en el sentido de que hay acciones de escritura y bytes no-cero. Pero todavía falta demostrar que la información escrita cause mejores decisiones. Para eso hacen falta evaluaciones como: congelar política y borrar bytes, permutar bytes entre celdas, bloquear escritura solo en test, igualar costos entre 4 y 8 bits, y comparar `mlp` contra `strided_cnn` con el mismo entorno.

La tercera limitación es de escala. El resultado 60-ant 50x50 es muy bueno, pero usa muchos agentes y escritura saturada. Puede ser la dirección de ingeniería correcta, pero no responde por sí solo la pregunta de comunicación compacta.

La cuarta limitación es de trazabilidad negativa. Algunas ramas importantes, como laberintos y cuadernos laterales de prueba de estrés, quedaron mejor representadas por código y commits que por artefactos finales de evaluación. En este informe se incluyen porque explican decisiones del proyecto, pero no se usan como evidencia cuantitativa de desempeño.

8. Conclusión

El proyecto produjo una historia clara. Primero hubo que corregir la semántica de la tarea: entregar comida, no solo encontrarla. Luego hubo que hacer aprendible el crédito: moldeado de distancia, más agentes y despliegue muestreado. Después apareció el detalle más importante para 50x50: el crítico centralizado espacial. Ese cambio reestructura el aprendizaje y debe figurar como una causa mayor. La memoria por bytes sigue siendo una hipótesis interesante, no una conclusión demostrada.

La contribución honesta del trabajo no es “resolvimos comunicación emergente”. Es más precisa: construimos un entorno de forrajeo multi-agente con memoria externa, identificamos qué cambios desbloquean cada escala, alcanzamos solución robusta en 25x25 y resultados muy fuertes en 50x50 bajo crítico espacial, y mostramos que en 250x250 el retorno moldeado puede engañar si no se mide entrega real.

A. Trabajo futuro

El informe cierra la historia experimental disponible en el checkout actual. Quedan, sin embargo, extensiones naturales para convertir varias lecturas en evidencia causal más fuerte:

1. congelar una tabla canónica de runs y checkpoints por familia;
2. agregar una tabla de hiperparámetros por familia, especialmente `critic_architecture`, `food_count`, `food_sources`, número de hormigas, bits y modo de evaluación;
3. correr ablaciones emparejadas mínimas para comunicación: sin escritura, bytes permutados y mismo costo entre 4 y 8 bits;
4. decidir si el flujo de laberintos se rerunnea como experimento comparable o queda fuera del conjunto empírico final.

Referencias

- [1] Jakob Foerster, Ioannis Alexandros Assael, Nando de Freitas, and Shimon Whiteson. Learning to communicate with deep multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, 2016.
- [2] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*, 2017.
- [3] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation, 2015.
- [4] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.
- [5] Sainbayar Sukhbaatar, Arthur Szlam, and Rob Fergus. Learning multiagent communication with backpropagation. In *Advances in Neural Information Processing Systems*, 2016.
- [6] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of ppo in cooperative multi-agent games. In *Advances in Neural Information Processing Systems*, 2022.

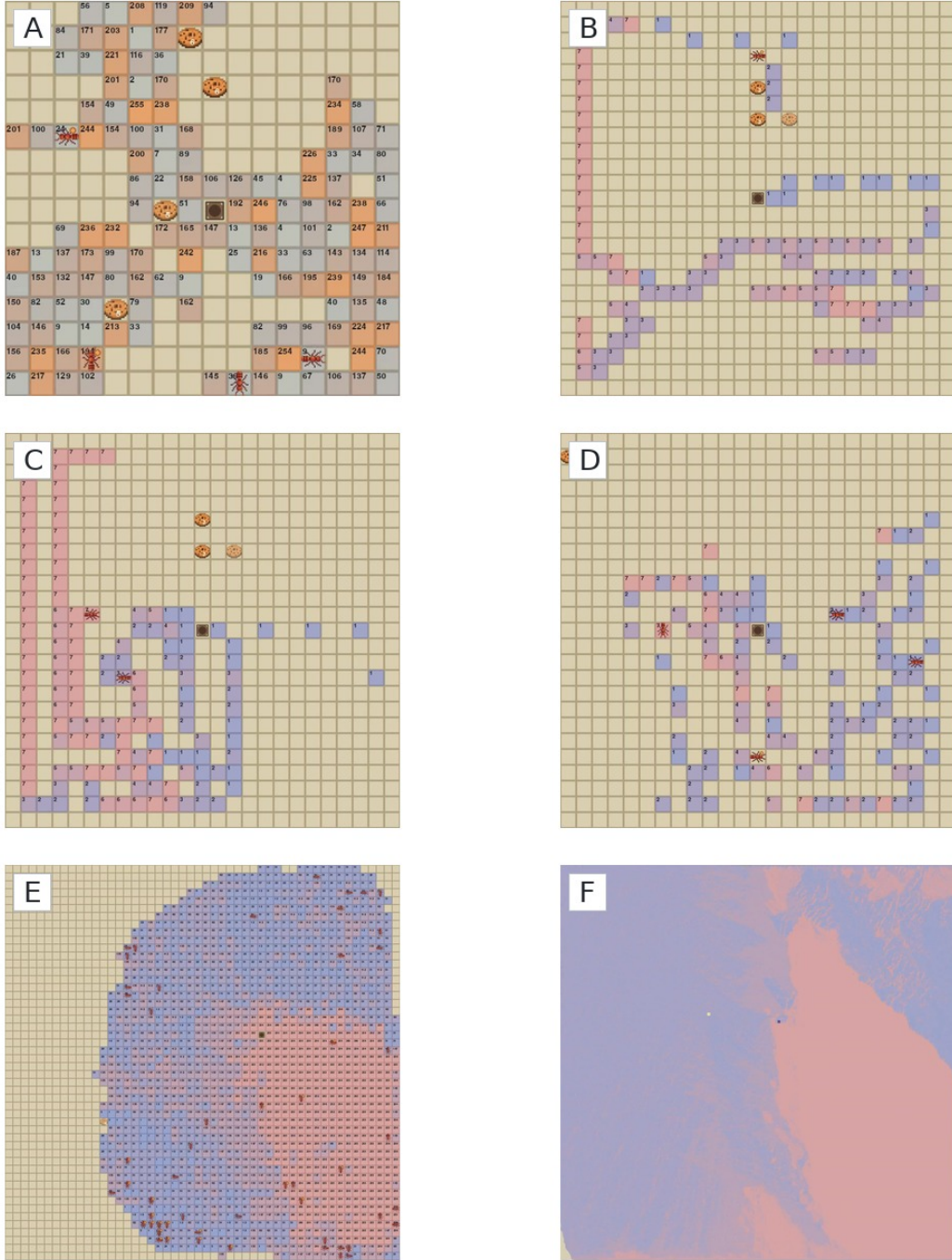


Figura 7: Fotogramas extraídos de videos locales. A: línea base aleatoria. B–D: progresión visual del currículo 25x25 con 2, 4 y 8 hormigas. E: frontera 50x50 con 60 hormigas. F: despliegue solo-actor 1000x1000 de la política 50x50. La figura no es evidencia cuantitativa; sirve para interpretar la semántica visual de las métricas.